Monolith vs Microservices

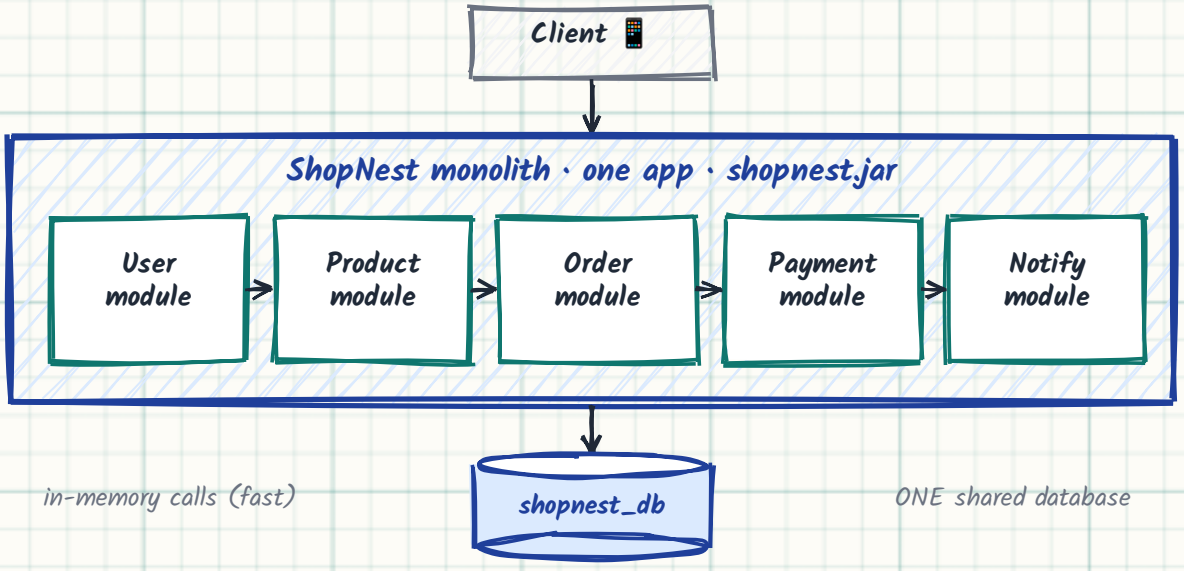
Lesson 1 · why microservices exist (and when they don't)

In this lesson → ① What is a monolith ② Where it hurts ③ What are microservices ④ Side-by-side comparison ⑤ The cost of microservices ⑥ When to choose what ⑦ Interview Q&A + cheat sheet

Running project: ShopNest, a small e-commerce app (users, products, orders, payments, notifications).

① What is a Monolith?

> **Monolith** = an application built & deployed as ONE single unit: one codebase, one process, one deployable file (e.g. shopnest.jar) and usually **one shared database**.



Why teams love it ✓

- ✓ **Simple to build:** one repo, one IDE project
- ✓ **Simple to test:** start one app, test end-to-end
- ✓ **Simple to deploy:** copy one file, run it
- ✓ **Fast calls:** module → module is an in-memory method call (ns)
- ✓ **Easy transactions:** one DB → one @Transactional

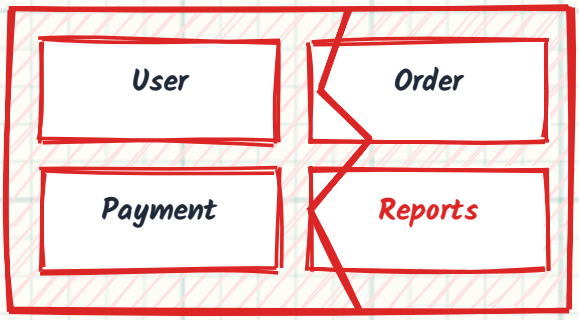
Remember!
Monolith ≠ bad. Most successful products **started** as monoliths. Problems appear when the app and the team grow.

Modular monolith
One deployable, but code split into clean modules with strict boundaries. The best **starting point** for most new products.

② When the Monolith starts to hurt

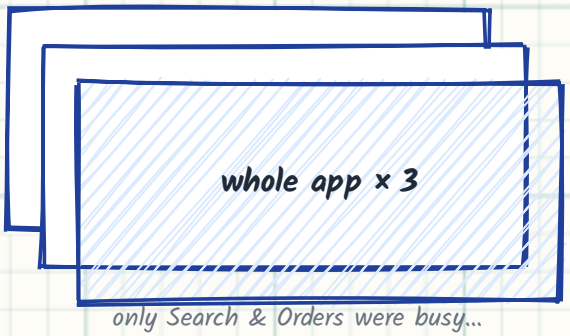
ShopNest is now a hit: **60 developers, 5 teams, 1 codebase**. Look what happens 👇

① one bug → everything down



memory leak in Reports = checkout down too

② scale ALL to scale one part



only Search & Orders were busy...
but you pay for 3 copies of everything 💸

1. **Slow & risky deployments:** change one button in Notifications → redeploy the entire app. Everyone waits for one big "release train".
2. **One bug can crash everything:** a memory leak in Reports takes down Checkout too → **single point of failure**.
3. **Can't scale parts independently:** on sale day only Search & Orders are busy, but you must copy the whole app.
4. **Tech lock-in:** one language, one framework version for everything. Upgrading Spring = upgrading all modules together.
5. **Team conflicts:** 5 teams in 1 repo → merge conflicts, broken builds, "who changed this?!"
6. **Huge codebase:** slow startup, slow builds, new joiners need weeks to understand it.

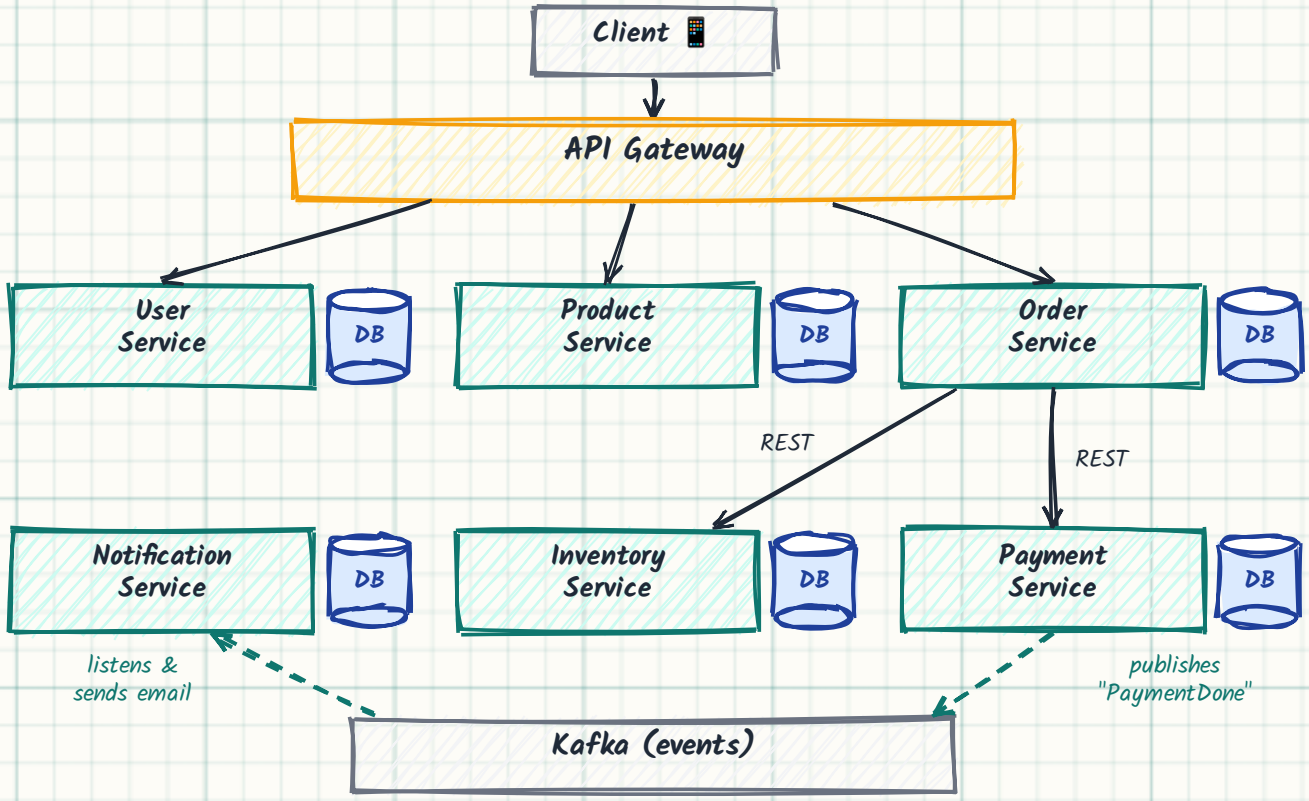
Key insight
The monolith's pain is mostly an **organisation & scaling** problem, not a code problem. Microservices are a way to let **many teams move independently**.

Real story: Netflix (2008) → a major database corruption stopped DVD shipping for about **three days**. That pushed them from one big monolith in their own data centre to **hundreds of microservices on the cloud**, a migration they finished around 2016.

Your turn ✍️
Think of the project you work on. Which 2 modules change most often? Which one gets the most traffic? Those are the first candidates to become separate services.

③ What are Microservices?

> **Microservices** = an architectural style where one application is built as a collection of small, independent services, each built around **one business capability**, running in its **own process**, owning its **own data**, and **deployed independently**.



Legend: solid arrow = sync call (REST/gRPC) · dashed arrow = async event (Kafka) · cylinder = database

6 characteristics (write these in interviews!)

- → **Single responsibility:** one business capability per service
- → **Independently deployable:** release Payment without touching Orders
- → **Owns its data:** database per service, no sharing tables
- → **Communicates over network:** APIs (REST/gRPC) or events (Kafka)
- → **Small team ownership:** "you build it, you run it" (Amazon's two-pizza teams)
- → **Tech freedom:** each service picks the best language / DB for its job

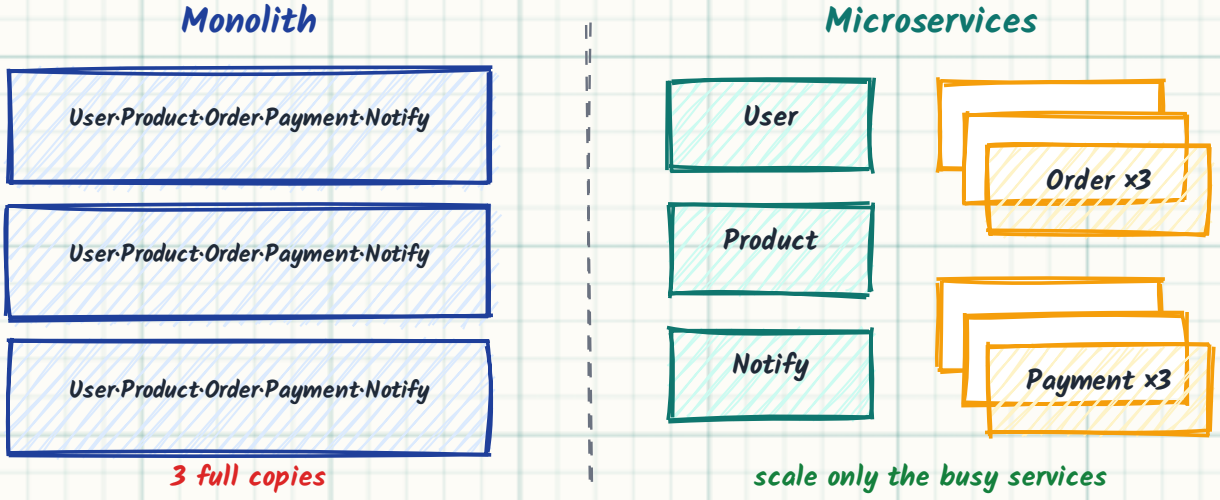
Analogy
Monolith = one big restaurant kitchen: one team cooks everything. If the kitchen floods, nobody eats.

...and microservices
Food court = many small stalls, each with its own staff & stock. Pizza stall closed? You can still get dosa.

④ Side-by-side comparison

Point	Monolith	Microservices
Deployable unit	One app (1 jar/war)	Many small apps, one per service
Database	One shared DB	DB per service
Communication	In-memory method calls (fast)	Network calls: REST / gRPC / events (slower, can fail)
Scaling	Scale the whole app	Scale only the busy service
Failure	One bug can take everything down	Failure isolated to one service (if designed well)
Deployment	Big, infrequent, risky	Small, frequent, independent
Transactions	Easy: one ACID transaction	Hard: Saga / eventual consistency
Tech stack	One stack for all	Polyglot: per service
Testing & debugging	Easy: one process	Hard: needs tracing, contract tests
Team fit	Small team, new product	Many teams, mature product & DevOps

Scaling picture: sale day on ShopNest 🛒



Microservices = pay for extra capacity only where the load is.

Myth vs Fact

- ✗ **"Microservices make the app faster."** Fact: each network hop adds latency; the win is **scaling & team speed**, not raw speed.
- ✗ **"Micro = tiny, like 100 lines."** Fact: size = one business capability, not lines of code.
- ✗ **"Using Docker/Kubernetes = microservices."** Fact: you can containerise a monolith too. Architecture ≠ tooling.

5 Microservices are NOT free

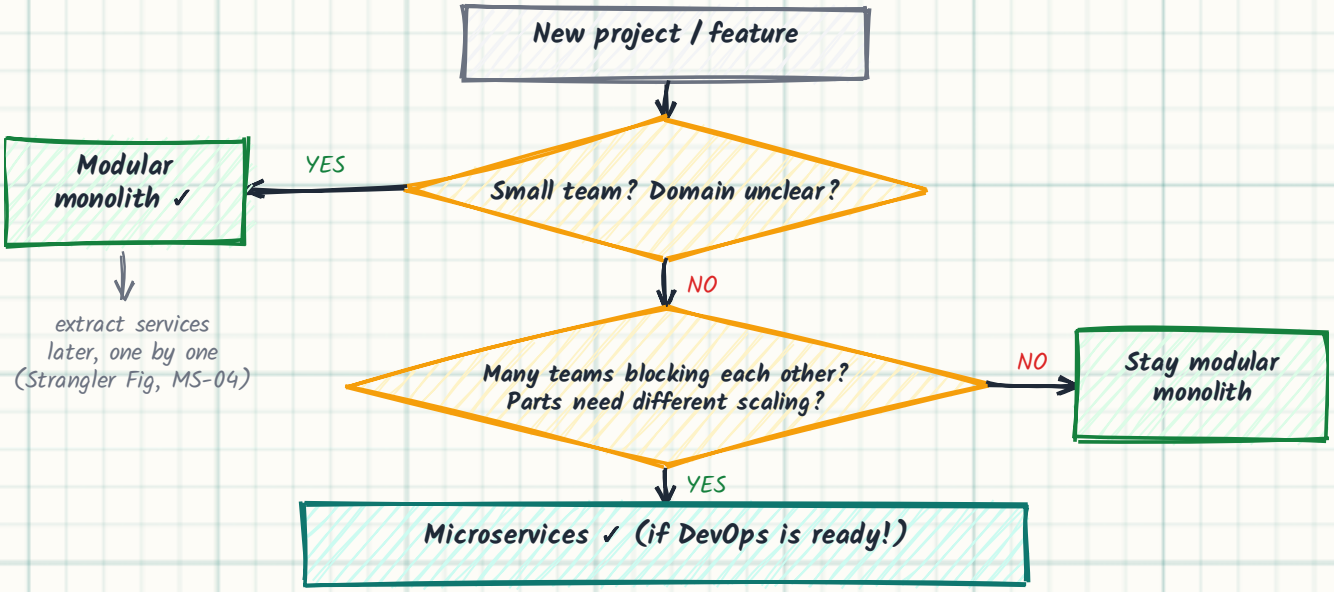
Most tutorials skip this. **Interviewers love it.**

- ✗ **Network calls fail:** a method call never times out; an HTTP call can → need timeouts, retries, circuit breakers (MS-14)
- ✗ **Distributed data:** no single @Transactional across services → Saga pattern (MS-11)
- ✗ **Operational complexity:** many pipelines, deployments, service discovery, gateway, monitoring (MS-08, 09, 19–21)
- ✗ **Hard debugging:** one request touches 5 services → distributed tracing (MS-22)
- ✗ **Extra latency:** every network hop adds milliseconds
- ✗ **Data consistency:** services see changes a little later (eventual consistency)

Real story: Amazon Prime Video (2023) → one monitoring component moved **back** from a distributed design to a single process and cut that component's infrastructure cost by about **90%**.

Microservices are a trade-off, not an upgrade!

6 When to choose what?



Microservices at scale

Netflix, Amazon: huge products, hundreds of teams, need independent scaling & deploys.

Modular monolith at scale

Shopify is well known for running a large **modular monolith** (Ruby on Rails). A monolith can scale too!

7 Interview Corner

Q. What is the difference between a monolith and microservices?

A. Compare on 5 points: **deployment unit** (one vs many), **database** (shared vs per service), **scaling** (whole app vs per service), **failure isolation**, and **team ownership**.

Q. What are the disadvantages of microservices?

A. Network failures & latency, distributed transactions, eventual consistency, operational complexity (CI/CD, monitoring, discovery), harder testing & debugging.

Q. Would you always choose microservices for a new project?

A. **No.** It depends on team size, domain clarity and scaling needs. For a new product, start with a **modular monolith** and extract services when there is a real reason.

Q. What is a "distributed monolith"?

A. Services that are split physically but still tightly coupled: they must be deployed together, share a DB, or call each other in long chains. You get **all the costs** of microservices and **none of the benefits**.

Q. How "micro" should a microservice be?

A. Size is not measured in lines of code. A service should match **one business capability** (bounded context) and be owned by **one team**. (Details in MS-03)

⚡ Revise in 60 seconds

- → **Monolith** = 1 app · 1 deploy · 1 DB
- → **Microservices** = many small services · own data · independent deploys
- → Monolith pain = **scale + big teams**
- → Microservices win = **independent deploys, scaling, failure isolation, team autonomy**
- → Microservices cost = **network, data consistency, ops, debugging**
- → Start with a **modular monolith**
- → Avoid the **distributed monolith**
- → Golden line: "trade-off, not upgrade"

Next lesson → MS-02
Characteristics of microservices & **5 signs you should NOT use them.**
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**